A Project Report
On
SMASH ZONE: AI SURVIVAL GAME

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of
Technology in Computer Science & Engineering-AIML

Submitted to:

RAJIV GANDHI PRODYOGIKI VISHWAVIDYALAYA, BHOPAL (M.P.)

Submitted by:
Srishti Choubey – 0808AL 251272
Siddhant Thakur – 0808AL251265

Under the Supervision of:
Mr. Deepak Shukla

IPS ACADEMY, INDORE

INSTITUTE OF ENGINEERING & SCIENCE
(A UGC Autonomous Institute Affiliated to RGPV)
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING-AIML
SESSION: 2026-27

CONTENTS

Section	Contents
Preliminary	Abstract; List of Figures; List of Tables; List of Abbreviations
Chapter 1	Introduction; Problem Statement; Objectives; Scope and Limitations
Chapter 2	AI Concepts and Techniques; Dataset Requirement
Chapter 3	Technologies; System Architecture; Methodology / Workflow
Chapter 4	Working Implementation; Algorithms; Main Game Features
Chapter 5	Results; Testing; Screenshots and Observations
Chapter 6	Conclusion and Future Scope
References	Project source file and technical documentation

Page numbers may shift slightly if the report is edited. Update the contents after final formatting if required by the department

ABSTRACT

Smash Zone is a browser-based survival game implemented using HTML, CSS, JavaScript, and the HTML5 Canvas 2D API. The player must survive against continuously spawning enemies, destroy them, collect power-ups, and achieve a high score. The game includes Auto Aim and Manual Aim modes, three difficulty settings, multiple enemy types, several weapons, bombs, shields, combo scoring, and a local scoreboard.

The main AI-related aspect of the current implementation is algorithmic Game AI. In Auto Aim mode, the program checks the positions of active enemies, selects the nearest one, calculates the direction using an angle function, and aims toward that target. Enemies also move toward the player's position using direct position and distance calculations. Enemy spawning and item drops include random choices, while the spawn rate and enemy speed can change with survival time. These features are controlled by programmed rules; the game does not train a machine-learning or deep-learning model.

Claude was used as an AI-assisted coding tool during development to help generate or modify code according to requested features. It is not the runtime AI engine controlling the game. The project demonstrates how basic decision rules, geometry, real-time updates, and visual rendering can be combined to build an interactive game. Because no model is trained, a training dataset is not required for the current version. Actual running-game screenshots should be added in the designated results section before final submission.

Keywords: Game AI, rule-based decision making, target selection, auto aim, enemy pursuit, JavaScript, Canvas API, dynamic difficulty.

LIST OF FIGURES

Figure No.	Title
Figure 3.1	Simplified system architecture of Smash Zone
Figure 3.2	Auto Aim decision cycle
Figure 5.1	Game menu screenshot — replace placeholder with a real capture
Figure 5.2	Gameplay screenshot — replace placeholder with a real capture

LIST OF TABLES

Table No.	Title
Table 2.1	AI concepts used in the current implementation
Table 3.1	Technologies and their roles
Table 4.1	Enemy types and their properties
Table 4.2	Weapon types
Table 5.1	Functional test checklist

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
UI	User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
HUD	Heads-Up Display
FPS	Frames Per Second
dt	Delta time: elapsed time between game updates

CHAPTER 1 — INTRODUCTION

1.1 Background of the Project

Interactive games combine input handling, game rules, graphics, movement, and feedback. In a survival game, the challenge comes from responding to enemies while managing health, weapons, and resources. Smash Zone implements these elements in a single HTML file that runs in a modern browser without a separate server or database.

The project also demonstrates a limited but meaningful form of Game AI. The computer-controlled enemies move toward the player, and Auto Aim can select the nearest enemy without requiring the player to point manually. These behaviors make the game more interactive, even though they are based on explicit algorithms rather than a model trained from examples.

1.2 Problem Statement

A survival game can become repetitive if its enemies and difficulty remain completely static. The objective of Smash Zone is to provide a browser-based game in which enemies can appear over time, pursue the player, and present a growing challenge. The game also needs to support target selection, shooting, collision handling, scoring, and resource-based actions in real time.

1.3 Objectives

- Develop a playable survival game that runs in a web browser.
- Implement rule-based Auto Aim by selecting the nearest active enemy.
- Implement enemy pursuit toward the player's current position.
- Provide different enemy types with different health, size, speed, and score values.
- Include weapons, bombs, shields, health recovery, and other power-ups.
- Increase challenge over time through spawning and speed adjustments.
- Display score, timer, health, combo, enemy count, and best score.
- Use the project to study how basic game logic and AI-style decision rules work together.

1.4 Scope

The current scope is a local, single-player, browser-based game. The source includes game rendering, player controls, enemy behavior, weapon logic, collision checks, score tracking, and local persistence. The current version does not include online multiplayer, a remote database, a trained AI model, or obstacle-based pathfinding.

1.5 Limitations

Enemy behavior is deliberately simple. Enemies move directly toward the player and do not plan routes around obstacles. Auto Aim uses nearest-target selection rather than tactical evaluation. The scoreboard is stored in the browser's localStorage, so it is local to that browser and device. The game does not learn from player behavior.

CHAPTER 2 — AI CONCEPTS AND TECHNIQUES

2.1 What type of AI is used?

The accurate description for the current project is algorithmic or rule-based Game AI. The code contains predefined conditions and calculations that produce automatic behavior. The term AI is used here in the broad game-programming sense of decision rules and autonomous actions. It should not be described as machine learning or deep learning, because the implementation contains no training process, model, or learned parameters.

2.2 AI concepts in the implementation

Concept	How it appears in Smash Zone	Important qualification
Rule-based decision ma	Pkriendgefined conditions determine when to ai	mR, ules, aspa wnitenemiics, dpplyda mage, thechanc eidvaftomredsaturces.
Nearest-target select	oTnhe program loops through active enemies a	dTchisesian simpleonte wgith-stheclimal hetusptiaced distance from the player.
Automatic aiming	The direction to the selected enemy is calcula	Atudowith Mathg,aurtah2nac,dnsotmeachisnthe eplayeng's angle.
Autonomous pursuit	Each enemy updates its position in the directi	Doinreocfttphuerpsulaiyt;enr.o A* or obstacle navigation.
Randomization	Random values influence spawn angle, enemy	Rtaypte,motesicals,npaticsleffcts,nalreasoing. drops.
Dynamic difficulty	Spawn intervals and enemy speed can chan	eThas adaptatiomfolowsaregdiffcultyfomtulas. modfy parameters.
State-based behavior	The game uses states such as menu, play, p	uAsefi,naitned-sotavteer-.machine-like structure is a useful description, though no

2.3 Nearest-enemy selection

The update function checks every active enemy. For each enemy, it computes the squared distance from the player: $dx^2 + dy^2$. If this value is smaller than the best value found so far, that enemy becomes the current target. Squared distance is sufficient for comparison because taking a square root is unnecessary when only relative distances are being compared.

```
d2 = (enemy.x - player.x)^2 + (enemy.y - player.y)^2
if d2 < nearestDistance:
    nearestDistance = d2
    target = enemy
```

If there are n active enemies, this selection scans the list once, so its time complexity is O(n).

2.4 Automatic aiming and angle calculation

Once the target is selected, the code calculates the angle from the player's position to the target using Math.atan2(target.y - player.y, target.x - player.x). The resulting angle is used to orient the player's weapon and direct projectiles. In Manual Aim mode, the angle is instead calculated from the player's position to the mouse pointer.

```
angle = Math.atan2(target.y - player.y,
                   target.x - player.x)
```

2.5 Enemy pursuit

For each enemy, the code calculates the horizontal and vertical differences between the player and enemy. It then calculates the distance and divides the direction components by that distance to obtain a normalized direction. Multiplying the normalized direction by enemy speed and elapsed time moves the enemy toward the player at a frame-rate-independent rate.

```
dx = player.x - enemy.x
dy = player.y - enemy.y
distance = Math.hypot(dx, dy) || 1
enemy.x += dx / distance * enemy.speed * dt
enemy.y += dy / distance * dt
```

2.6 Dynamic difficulty and randomness

The spawn function chooses an angle and places enemies outside the central play area. As time passes, the probability of spawning fast or tank enemies can increase according to the conditions in the code. The spawn interval is also adjusted over time, subject to a minimum interval and an active-enemy limit. These are programmed difficulty rules, not a learned policy.

2.7 Hallucination, illusion, ML, and DL

AI hallucination is commonly discussed in generative AI systems that may produce confident but false information. Smash Zone does not generate factual answers, so hallucination is not a technique used by the game. The enemies may create an illusion of intelligence because they automatically chase the player, but this is an informal description of the effect rather than a separate implemented AI algorithm. Machine learning, deep learning, and reinforcement learning are not implemented in the current code.

2.8 Dataset requirement

No training dataset is required for the current version because the game does not train a model. The game uses the current positions, timers, health values, and other runtime state to make rule-based decisions. A future machine-learning extension would need a clearly defined task and appropriate training or interaction data.

CHAPTER 3 — TECHNOLOGIES AND METHODOLOGY

3.1 Technologies used

Technology	Role in the project
HTML5	Provides the page structure, canvas element, overlay, and buttons.
CSS3	Styles the interface, menu, buttons, typography, colors, and responsive layc ut.
JavaScript	Implements player movement, enemies, aiming, weapons, collision, score, t mers, and game states.
HTML5 Canvas 2D API	Draws the arena, player, enemies, bullets, particles, effects, and HUD.
Browser localStorage	Stores best score and the top-five local scoreboard between visits, when brc wser storage is availabl
Claude AI	Used as an AI-assisted development tool for generating or modifying code; it does not control the ga

3.2 System architecture

The application is client-side. User input is read by browser event handlers. The JavaScript update function changes the game state, then the drawing function renders the current state on the canvas. The game loop schedules the next frame. Browser localStorage is used only for local score persistence.

Layer	Responsibility
User input	Keyboard, pointer, mouse, touch
Game logic	State, player, enemies, bullets, timers, collision, score
Decision rules	Nearest target, auto aim, enemy pursuit, spawn and difficulty rules
Rendering	Canvas 2D drawing and HTML/CSS overlay
Persistence	Browser localStorage for best score and scoreboard

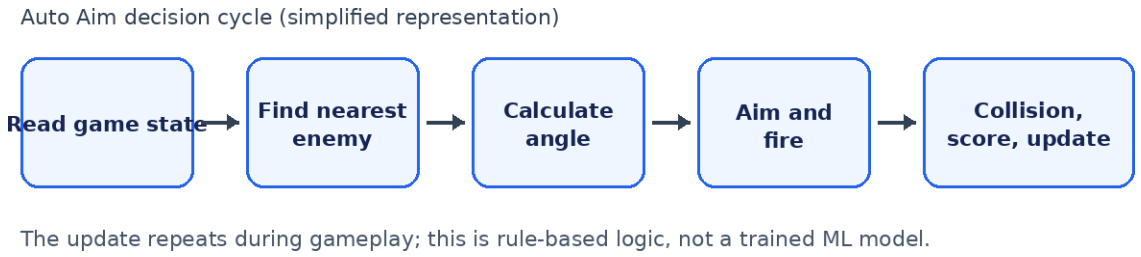


Figure 3.1 / 3.2. Simplified Auto Aim workflow created from the code structure.

3.3 Methodology / workflow

- The browser loads the HTML document and creates the canvas and interface elements.
- JavaScript initializes variables for player, enemies, bullets, effects, score, difficulty, and aim mode.
- Event listeners capture keyboard and pointer input.
- During each frame, the update function moves the player, updates timers, chooses a target, aims or reads manual input, fires when conditions permit, and spawns enemies.
- Bullets and enemies are checked for collisions; health, score, combo, and power-ups are updated.
- The drawing function renders the current game state on the canvas.
- The loop requests the next animation frame and repeats until the game is paused or over.

3.4 Game loop and delta time

The game uses requestAnimationFrame(loop) to schedule rendering with the browser's animation system. The loop calculates dt, the elapsed time in seconds since the previous frame, and caps it at 0.033 seconds. Movement and timers use dt so that behavior depends more on elapsed time than on the number of frames drawn.

```
var dt = Math.min(0.033, (now - last) / 1000);
if (S == "play") up(dt);
draw();
requestAnimationFrame(loop);
```

3.5 State management

The state variable S distinguishes menu, play, pause, and over behavior. For example, the update function runs only while S is play. Pausing displays an overlay and prevents normal gameplay updates; game over records the score and presents the result screen.

CHAPTER 4 — WORKING IMPLEMENTATION

4.1 Player and controls

The player is represented by a JavaScript object containing position, current and maximum health, invulnerability timer, firing timer, aim angle, bomb count, shield count, shield duration, weapon selection, weapon timer, and visual flash timer. Keyboard input supports WASD and arrow keys. Pointer dragging can also contribute movement. In Manual Aim mode, the mouse pointer determines aim direction and holding the left mouse button or the F key enables firing.

4.2 Enemy types

Type	Radius	Health	Base speed	Score value	Description
Normal	13	1	60	10	Standard enemy with balanced movemei t.
Fast	9	1	115	15	Smaller and faster enemy.
Tank	25	4	38	40	Larger, slower enemy that needs multip e hits.

The values above are taken from the enemy configuration in the source code. The speed applied during play is further scaled by time and the selected difficulty.

4.3 Weapons

Weapon	Behavior in code
Pistol	Default weapon; firing interval and projectile behavior depend on the weapon s ttings and score thresho
Rapid SMG	Short firing interval with a single projectile per shot.
Shotgun	Fires several projectiles with spread.
Piercing Rifle	Uses a higher-damage projectile that can pass through a limited number of ene ies.

4.4 Bombs, shields, and power-ups

A bomb consumes one available bomb and reduces each active enemy's health by three points; enemies with zero or less health are removed and scored. A shield consumes one shield charge and protects the player for five seconds. Drops can include health, weapon upgrades, bombs, or shields. A health pickup restores one health point without exceeding maximum health. Weapon pickups temporarily set the selected weapon for 14 seconds.

4.5 Collision detection

Bullet-enemy collision uses the distance between the bullet and enemy centers. If the distance is smaller than the enemy radius plus the bullet's collision radius, the enemy takes damage. Enemy-player collision checks whether the distance is below the enemy radius plus the player's collision allowance. A collision can reduce player health unless invulnerability or a shield is active.

```
if (Math.hypot(enemy.x - bullet.x,
               enemy.y - bullet.y) < enemy.r + 5) {
    enemy.hp -= bullet.damage;
}
```

4.6 Scoring, combo, and scoreboard

When an enemy is defeated, the game adds the enemy's base score multiplied by a combo multiplier. The combo increases with consecutive kills and resets after its timer expires. At game over, the result is added to the scoreboard, which is sorted by score and limited to five entries. The best score and scoreboard are saved in localStorage when storage is available.

4.7 Rendering and visual effects

Canvas drawing functions render the background grid, enemies, bullets, player, weapon, shield, bomb wave, particles, and heads-up display. Particle bursts and screen shake provide feedback for hits and explosions. CSS styles the menu overlay, title, buttons, scoreboard, and controls. These visual effects improve feedback but are not themselves AI techniques.

4.8 Important implementation notes

The game is delivered as one HTML file containing CSS and JavaScript. No backend API, remote database, external ML library, or model file is present in the supplied source. The scoreboard persists in the browser rather than being shared online. Claude assisted with code creation/modification, but the program's in-game decisions are made by the JavaScript rules described in this report.

CHAPTER 5 — RESULTS, TESTING AND OBSERVATIONS

5.1 Results

The supplied source code contains the game menu, Auto Aim and Manual Aim modes, difficulty selection, player controls, enemy spawning and pursuit, shooting and collision logic, power-ups, scoring, game-over handling, and a local scoreboard. The table below is a code-based functional checklist. It describes intended behavior supported by the implementation; it should not be presented as a formal benchmark or a record of independently measured performance.

Test	Expected behavior from the code	Status basis
Open HTML file	Menu and canvas are initialized.	Source inspection
Choose difficulty	Game starts with selected difficulty settings.	Source inspection
Move player	Keyboard and pointer controls update player position.	Source inspection
Auto Aim	Nearest active enemy is selected and aimed at.	Source inspection
Manual Aim	Aim follows mouse; firing responds to input.	Source inspection
Hit enemy	Bullet collision applies damage; defeated enemy adds score.	Source inspection
Enemy reaches player	Health decreases unless shield/invulnerability protects player.	Source inspection
Use bomb / shield	Bomb damages enemies; shield provides timed protection.	Source inspection
Collect power-up	Health, weapon, bomb, or shield count/effect updates.	Source inspection
Game over	Scoreboard and best score are updated in localStorage when avai	Source inspection

5.2 Screenshots

The supplied report sample does not contain Smash Zone screenshots, and the source file alone does not provide captured output images. The panels below are clearly marked placeholders, not screenshots of a live game. Before submission, replace each panel with a genuine screenshot captured while running the supplied HTML file in a browser. Recommended captures: (1) menu showing aim modes and difficulty choices; (2) active gameplay showing the player, enemies, score, timer, health, and aim mode; (3) game-over screen with score and scoreboard.

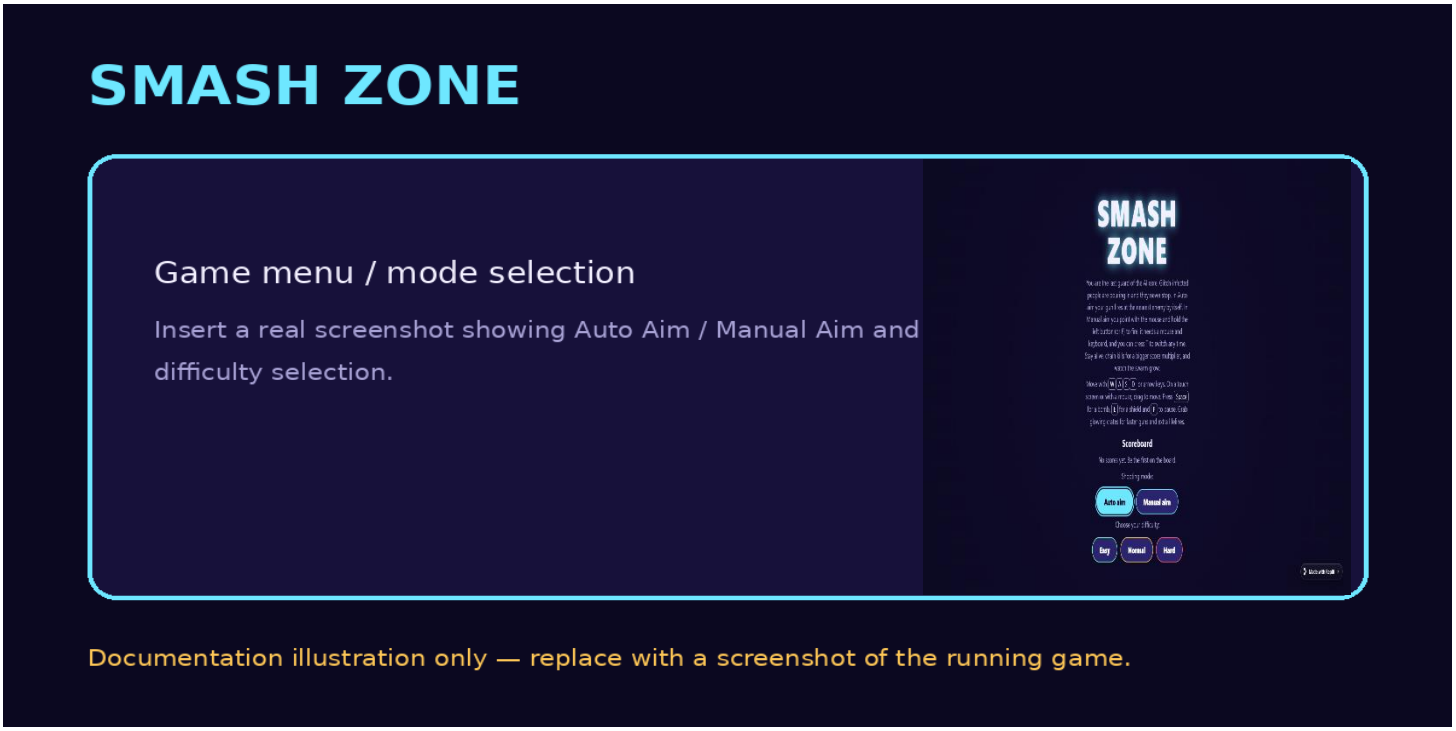


Figure 5.1. Screenshot placeholder — replace with an actual capture of the game menu.



5.3 Observations

- The nearest-enemy scan is linear in the number of active enemies, O(n), because it visits each enemy once.
- Enemy pursuit is direct and does not calculate routes around obstacles.
- The game caps the number of active enemies at 80 in the spawning condition.
- Difficulty behavior is based on preset settings and time-dependent rules, not player learning.
- localStorage provides local persistence only; scores are not synchronized across devices.

5.4 Limitations identified

The project has no trained ML model, no dataset, no online leaderboard, no server-side persistence, no multiplayer, and no obstacle-aware pathfinding. Random enemy spawning can make runs vary, but the program does not learn from previous runs. Any future ML feature should be described as future work until it is implemented and tested.

CHAPTER 6 — CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

Smash Zone demonstrates how a small browser game can combine real-time interaction, graphics, programmed decision rules, and simple Game AI behavior. Its main AI-style features are nearest-target selection, automatic aiming, direct enemy pursuit, randomized spawning, and time-dependent difficulty. The game also brings together movement, weapons, collision detection, power-ups, scoring, visual effects, and local score persistence in a single HTML implementation.

The most important learning outcome is understanding the difference between rule-based Game AI and machine learning. In the current implementation, the system follows predefined logic; it does not train on examples or improve through reinforcement. Claude was used as an AI-assisted development tool, while the browser executes the game's JavaScript logic. The report therefore describes the actual techniques present instead of claiming unsupported ML or DL functionality.

6.2 Future scope

- Introduce obstacle-aware navigation, potentially using A* if the arena includes walls or barriers.
- Explore reinforcement learning for an enemy policy, with a defined reward function and an evaluation setup.
- Collect gameplay telemetry and investigate whether a machine-learning model could classify player behavior or estimate difficulty; this would require suitable data and testing.
- Create more enemy behaviors, such as ranged enemies, bosses, or enemies that coordinate.
- Add an online leaderboard through a backend service, with appropriate validation and security.
- Improve accessibility, mobile controls, settings, sound, and performance testing.

6.3 Learning outcomes

The project provides practice in JavaScript objects and arrays, event-driven programming, coordinate geometry, distance and angle calculations, time-based movement, collision detection, animation loops, UI design, and local browser storage. It also demonstrates why technical reporting should distinguish implemented features from possible future extensions.

REFERENCES

1. Smash Zone source file supplied for this report: “Smash Zone_ AI Survival Game.html”.

2. MDN Web Docs, “Canvas API” and “Window: requestAnimationFrame()” — used as general references for browser graphics and animation concepts.

3. MDN Web Docs, “Window.localStorage” — general reference for browser-local data persistence.

4. Project development notes: Claude was used as an AI-assisted coding tool; the actual implementation details were checked against the supplied HTML source.

Note: The screenshots section still contains explicit placeholders because no genuine gameplay screenshots were included with the supplied files. Replace them with captures from the running project before submitting the report.